

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260619_051348_098084 -->

Framework: JUnit | Run ID: run_20260619_051348_098084

Tóm tắt kỹ thuật

- Framework: JUnit 5 (JUnit Jupiter)
- Kết quả chạy: Tất cả 15 test pass, không lỗi biên dịch hay runtime.
- Coverage: 96.27 % (đạt quality gate “coverage”).
- Issue type: Không có lỗi thực thi (issue_type: none).
- Bản kiểm thử được xem xét: BankAccountTest.java
- Mã nguồn được kiểm thử: BankAccount.java

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- Thiếu kiểm thử cho getter isActive() ngay sau khi tạo tài khoản.
- Bằng chứng: Test plan TC-001 và TC-005-TC-015 không có bước xác nhận account.isActive() trả về true khi tài khoản mới tạo.
- Ảnh hưởng: Không kiểm chứng yêu cầu “Calling isActive() returns true if the account is active”.
- Thiếu kiểm thử cho hành vi getBalance() khi tài khoản không hoạt động.
- Bằng chứng: Yêu cầu “Expected behavior for getBalance() when the account is inactive” được liệt kê trong *missing_information* nhưng không có test nào.
- Ảnh hưởng: Không biết liệu getBalance() có bị hạn chế khi tài khoản không hoạt động hay không, gây rủi ro khi thay đổi logic.
- Thiếu kiểm thử cho constructor cho phép thiết lập trạng thái isActive (nếu có).
- Bằng chứng: Test plan có các TC yêu cầu tạo tài khoản với isActive=false (TC-007, TC-011, TC-015) nhưng lớp BankAccount không có constructor nhận tham số này; các test mô phỏng bằng cách deactivate sau khi rút hết số dư.
- Ảnh hưởng: Nếu trong tương lai có thêm constructor cho phép khởi tạo tài khoản không hoạt động, các test hiện tại sẽ không phát hiện lỗi.

Assertion yếu

- Không phát hiện.
- Tất cả các test đều sử dụng assertEquals, assertThrows, hoặc assertFalse với thông điệp mô tả, không có assertion luôn đúng hoặc chỉ in log.

Rủi ro maintainability

- Không phát hiện.
- Các test không phụ thuộc vào thời gian, mạng, hay hệ thống file.

- Không có mocking; mọi thao tác thực hiện trên đối tượng thực.

Gợi ý sửa ngay

1. Thêm test xác nhận trạng thái hoạt động ngay sau khi tạo tài khoản.

@Test

```
void testIsActiveAfterCreation() {  
    BankAccount account = new BankAccount("ACC999", 0.0);  
    assertTrue(account.isActive(), "Tài khoản mới tạo phải ở trạng thái hoạt động");  
}
```

2. Thêm test kiểm tra getBalance() khi tài khoản đã bị deactivate.

@Test

```
void testGetBalanceWhenInactive() {  
    BankAccount account = new BankAccount("ACC000", 50.0);  
    account.withdraw(50.0);  
    account.deactivate();  
    assertFalse(account.isActive());  
    assertEquals(0.0, account.getBalance(), 0.001, "Số dư khi tài khoản không hoạt động vẫn trả về giá trị hiện tại");  
}
```

3. Nếu trong tương lai sẽ có constructor cho phép chỉ định isActive, viết test cho trường hợp tạo tài khoản không hoạt động và xác nhận các hành vi (deposit, withdraw, transfer) ném IllegalStateException.

@Test

```
void testConstructorWithInactiveFlag() {  
    BankAccount account = new BankAccount("ACC_INACTIVE", 0.0, false); // giả sử có constructor mới  
    assertFalse(account.isActive());  
    assertThrows(IllegalStateException.class, () -> account.deposit(10.0));  
}
```

(Lưu ý: chỉ thực hiện khi API thực tế được bổ sung.)
